Handover & Testing Guide

AES Operations & Finance Management System

A plain-English guide to what the system does, how to start it, and how to move around it — written so the next person can pick it up, test it, and use it.

Prepared 20 July 2026 · Airflow Environmental Solutions (AES) · Repository: github.com/Vulture-Nest/aes-platform

1. What is this system, in plain terms?

This is the internal software for **Airflow Environmental Solutions (AES)**, a Zimbabwean mining-services contractor. It replaces the spreadsheets the business used to run on. In one place it handles **who can do what**, **money (in USD and Zimbabwe Gold)**, **approvals for spending**, the **financial records**, **payroll**, a **business-health dashboard with early-warning alerts**, and a **sales pipeline (CRM)**.

Three ideas run through the whole system:

- **Approve first, fund second.** A spending request is approved on its merits; whether there's cash to pay it is a separate step that only changes its status (“Ready to Pay” vs “Pending Funds”). Money is never a reason an approval is blocked.
- **A person always moves the money.** The system prepares, approves and records; a human does the actual bank/wallet transfer and types in the reference.
- **Everything is recorded.** Every login, approval and change is written to an audit trail that can't be edited.

2. What can it actually do?

Area	What it does (in plain terms)
Users & access	Log in securely; each person has one or more roles, per site (e.g. “Finance Officer at Mimosa”). The system only lets you see and do what your role allows.
Reference data	Exchange rates (official & street), tax/statutory rates, spending thresholds, and stand-in (delegation) rules for approvers who are away.
Approvals engine	One engine drives all approvals (requisitions, travel, petty cash, budgets, director withdrawals). You configure *who* approves *what* and in what order — no programming needed. Nobody can approve their own request.
Financial core	Clients, contracts, orders, receipts, expenses, loans, a double-entry accounts ledger, and tax records. All money is stored with its currency and exchange rate, never as a bare number.
Spending workflows	Cash requisitions, travel allowances, petty cash, budgets, and director withdrawals — each with its own approval path and a funds check.
Command Centre	A live business-health dashboard (cash position, money in vs out, debt, receivables, tax exposure, coverage ratio, a HEALTHY/WATCH/ACT verdict) plus an automatic *danger engine* that raises alerts and notifies directors when something crosses a limit.
Payroll	Timesheets → pay runs → statutory deductions (PAYE, NSSA, ZIMDEF, NEC/MIPF…) → payslips, bank schedules and a Sage journal. Bank account numbers are masked; payroll is restricted to finance roles.
CRM	Business-development contacts, a sales pipeline (Contact → Qualified → Proposal → Negotiation → Won/Lost), and one-click conversion of a won deal into a real order.
Settings	An admin area to manage the lists the system uses (currencies, roles, site types, statutory keys, etc.) without needing a developer.

3. The parts of the system

Part	Who it's for	Where it runs (locally)
API (the engine / server)	Not used directly by people; the two apps talk to it.	`http://localhost:3000` · interactive API docs at `/docs`

Admin Console	Administrators & finance — to set everything up and configure it.	`http://localhost:5174`
Web App	Everyday users — raise requests, approve, view dashboards.	`http://localhost:5173`
Mobile app	Site staff (planned).	Not built yet

4. Getting it running (for the person setting up the test)

There isn't a hosted website yet, so a technically-comfortable person starts it on a laptop. You'll need **Docker** and **Node.js 20+** installed. Open a terminal in the project folder and run these, one block at a time.

```
# 1) Start the database & support services (Docker)
docker compose -f docker-compose.dev.yml up -d
#    If ports 5432 / 6379 / 9000 are already in use on your machine, remap them:
#    POSTGRES_PORT=5433 REDIS_PORT=6380 MINIO_API_PORT=9002 MINIO_CONSOLE_PORT=9003 \
#      docker compose -f docker-compose.dev.yml up -d   (then use port 5433 in DATABASE_URL below)

# 2) The API (backend). In apps/api:
cp .env.example .env          # then open .env and set JWT_SECRET to any 16+ character phrase
npm install
npx prisma migrate deploy     # creates all the database tables
npx prisma db seed            # creates the sites, ledger accounts and the admin login
npm run start:dev             # serves http://localhost:3000  (docs at /docs)

# 3) The Admin Console (new terminal). In apps/admin:
npm install && npm run dev      # http://localhost:5174

# 4) The Web App (new terminal). In apps/web:
npm install && npm run dev      # http://localhost:5173
```

> **Quick health check:** open `http://localhost:3000/health` — it should say `"status":"ok"`. The full list of every function the system exposes is browsable at `http://localhost:3000/docs`.

5. Logging in

A fresh set-up (step 2 above) creates one master account. Use it first, then create everyone else from the Admin Console.

Administrator (full access)	`admin@aes.local` / `ChangeMe!123`
Finance Director (demo account on the current test database)	`fd@aes.local` / `FdPassword!123`

> **Before real use:** create proper user accounts and change these demo passwords. The administrator creates users under *Users & Roles* and assigns each person a role (and a site, where relevant).

6. Admin Console — the menu, explained

Sign in at `http://localhost:5174` as the administrator. The left-hand menu:

Menu item	What you do here
Dashboard	A quick overview (how many users, sites, exchange-rate entries).
Users & Roles	Create people, and give them roles (per site). This is where you add a Finance Officer, Site Manager, etc.
Sites	The company's sites (Mimosa, Unki, Zimplats, Head Office). Add or edit them.
Exchange Rates	Record the USD→ZWG rate for a date (official and street). History is kept — older rates are never overwritten.
Statutory Rates	Tax and payroll figures (VAT %, PAYE bands, NSSA, etc.), each effective from a date. Confirm real figures with a tax practitioner.
Thresholds	Tunable limits — e.g. the petty-cash amount above which a Finance Director must approve.
Delegation	Set a stand-in approver for a date range (when someone is on leave).

Approval Matrix	The rules for *who approves what*: pick a module (e.g. requisition), an amount band, the approver role, the step order, and the mode (one-after-another, all-together, or either).
Danger Rules	The automatic early-warning rules (e.g. cash runway below 4 weeks). Switch each on or off.
Ledger Accounts	The bank / petty-cash / mobile-wallet accounts money is tracked against.
Employees	Staff records used by payroll (bank details are masked).
Audit Log	A searchable record of everything that happened — who did what, and when.
Settings	The configurable lists: currencies, roles, site types, statutory keys, employment types, etc. Add a value here (e.g. a new currency) and it becomes usable everywhere.

7. Web App — the menu, explained

Sign in at `http://localhost:5173`. This is the everyday app. The menu:

Menu item	What you do here
Home	Your landing page with quick tiles and an unread-notifications badge.
Requests	Raise a **cash requisition** (purpose, amount, currency, needed-by date) and submit it for approval. See the status of your requests.
Approvals	Your **inbox**: items waiting for your decision, matched to your role. Approve, return for correction, or reject. (You can't approve something you raised.)
Orders	The order book, with each order's servicing status.
Command Centre	The 8-panel business-health dashboard with the overall HEALTHY / WATCH / ACT verdict.
Exchange Rates	View the current and historical rates (read-only here).
Notifications	Your alerts; mark them read.
Profile	Your account details and roles.

8. A five-minute test you can try (the “happy path”)

1. **As the administrator** (Admin Console): under *Users & Roles*, confirm there's a Finance Director (or use the demo `fd@aes.local`). Under *Approval Matrix*, confirm there's a rule for *requisition → Finance Director* (there is one by default).
2. **As a normal user** (Web App): go to *Requests → New requisition*, fill it in and *Submit*. Its status becomes *Submitted*.
3. **As the Finance Director** (Web App): go to *Approvals*. The requisition is in the inbox. Click *Approve*.
4. The system runs a funds check: it becomes *Approved – Ready to Pay* if there's cash, or *Approved – Pending Funds* (with the shortfall) if not.
5. Open the **Command Centre** to see the business-health panels update.

> **What this proves:** the approval engine, the “approve-first / fund-second” rule, the no-self-approval safeguard, and the live dashboard are all working together.

9. What's built vs. what's still to come

Built & working now	Not yet / needs input
• Full backend engine (150+ functions) with an automated test suite • Admin Console (all configuration) • Web App (requests, approvals, orders, command centre, notifications) • Approvals, financial core, workflows, command centre & alerts, payroll, CRM • All reference lists configurable from Settings	• **Mobile app** — planned, not built • **Excel import** of the old workbooks — needs the real Operational Cashflow & payroll files • **Power BI / Microsoft Fabric** analytics — needs a Fabric tenant • **Hosted deployment** (a URL for testers) — the automated deploy is prepared but not switched on • A few outputs are data-only for now (e.g. payslip *PDF* rendering is a later step)

10. For the developer taking over

- **Repository:** `github.com/Vulture-Nest/aes-platform` (private). One monorepo: `apps/api` (server), `apps/admin`, `apps/web`, `apps/mobile` (placeholder), `deploy/` (CI/CD + server config).
- **How it's built:** the work landed as a series of pull requests into `main` (foundations → approvals → financial core → workflows → command centre → payroll → CRM → settings).

- **API reference:** run the API and open `/docs` for the live, clickable list of every endpoint.
- **Tests:** `cd apps/api && npm test` (500+ unit tests). Each app also has `npm run build` / `lint`.
- **Deploy target:** Docker images → GitHub Actions → a DigitalOcean droplet with managed PostgreSQL. The workflow files live in `deploy/github-workflows/` and need to be moved to `.github/workflows/` with a "workflow"-scoped token to switch CI/CD on (see `deploy/README.md`).

11. Housekeeping before real use

- Change the seeded/demo passwords and create real user accounts with the right roles.
- Enter the *real* statutory figures (VAT, PAYE bands, NSSA…) under Statutory Rates — confirm them with a tax practitioner. Nothing is hard-coded.
- Rotate any access token that was shared during development.
- When ready, switch on the automated deployment so testers get a shared web address instead of running it locally.